

運用 Cloud Run 工作、BigQuery 和 Gemini，擴充 Video Insights Pipeline

來源：<https://codelabs.developers.google.com/video-insights-with-cloud-run-jobs?hl=zh-tw>

步驟數：12

在本程式碼研究室中，我們將展示可擴充的無伺服器解決方案，運用 Google Cloud 強大的服務套件 (Cloud Run、BigQuery 和 Google 的生成式 AI (Gemini)) 處理影片內容。

目錄

1. [總覽](#)
2. [事前準備](#)
3. [資料庫/倉儲設定](#)
4. [資料擷取](#)
5. [建立影片洞察函式](#)
6. [管道應用程式開發 \(Python\)](#)
7. [設定 Dockerfile 和容器化](#)
8. [建立 Cloud Run 工作](#)
9. [執行及監控 Cloud Run 工作](#)
10. [結果分析](#)
11. [清除所用資源](#)
12. [恭喜](#)

1. 總覽

在當今資料豐富的世界，從非結構化內容 (尤其是影片) 擷取實用洞察資料，是相當重要的需求。試想您需要分析成百上千個影片網址、歸納內容、擷取重要技術，甚至為教材生成問答組合。逐一執行不僅耗時，效率也不高。這正是現代雲端架構的優勢所在。

在本實驗室中，我們將逐步說明如何使用 Google Cloud 強大的服務套件 (Cloud Run、BigQuery 和 Google 的生成式 AI (Gemini))，處理影片內容的可擴充無伺服器解決方案。我們將詳細說明從處理單一網址，到協調大型資料集平行執行的過程，完全不需要管理複雜的訊息佇列和整合作業。

挑戰

我們負責處理大量影片內容，特別是實機實驗室課程。目標是分析每部影片，並生成結構化摘要，包括章節標題、簡介內容、逐步說明、使用的技術，以及相關的問答集。這項輸出內容必須有效儲存，以便日後用於建構教材。

一開始，我們有一個簡單的 HTTP 型 Cloud Run 服務，一次只能處理一個網址。這非常適合用於測試和臨時分析。不過，當您要處理從 BigQuery 取得的數千個網址清單時，這種單一要求、單一回應模型的限制就顯而易見。如果依序處理，可能需要數天甚至數週。

機會：將手動或緩慢的循序程序，轉換為自動化平行工作流程。我們希望透過雲端服務達成下列目標：

- 平行處理資料：大幅縮短大型資料集的處理時間。
- 運用現有 AI 功能：利用 Gemini 的強大功能進行精密的內容分析。
- 維護無伺服器架構：避免管理伺服器或複雜基礎架構。
- 集中管理資料：使用 BigQuery 做為輸入網址的單一可靠來源，以及處理結果的可靠目的地。
- 建構穩固的管道：建立可抵禦故障的系統，並輕鬆管理及監控。

目標

使用 Cloud Run 工作調度管理平行 AI 處理作業：

我們的解決方案以 Cloud Run 工作為中心，做為自動化調度管理工具。這項服務會從 BigQuery 智慧讀取批次網址、將這些網址傳送至現有已部署的 Cloud Run 服務 (負責處理單一網址的 AI 處理作業)，然後彙整結果並寫回 BigQuery。這種做法可讓我們：

- 將自動化調度管理與處理作業分離：工作負責管理工作流程，而獨立的服務則專注於 AI 任務。
- 善用 Cloud Run Job 的平行處理功能：這項工作可以水平擴展多個容器執行個體，同時呼叫 AI 服務。
- 降低複雜度：我們讓工作直接管理並行 HTTP 呼叫，實現平行處理，簡化架構。

用途

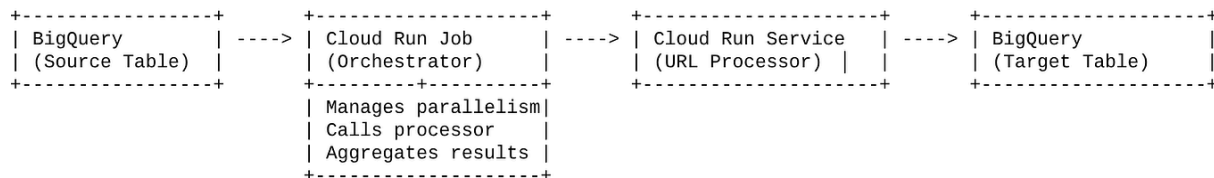
從 Code Vipassana 課程影片取得 AI 輔助洞察資料

我們的具體用途是分析 Code Vipassana 實作實驗室的 Google Cloud 課程影片。目標是自動生成結構化文件 (包括書本章節大綱)，內容如下：

- 章節標題：每個影片片段的簡潔標題
- 簡介背景資訊：說明影片在更廣泛的學習路徑中的關聯性
- 要建構的內容：工作階段的核心任務或目標
- 使用的技術：列出提及的雲端服務和其他技術
- 逐步說明：如何執行工作，包括程式碼片段

- 原始碼/試用版網址：影片中提供的連結
- 問答環節：生成相關問題和答案，以檢查知識。

Flow



架構流程

什麼是 Cloud Run？什麼是 Cloud Run 工作？

Cloud Run

這個全代管無伺服器平台可讓您執行無狀態容器，非常適合網頁服務、API 和微服務，可根據傳入的要求自動調整資源配置。您提供容器映像檔，Cloud Run 會處理其餘事項，包括部署、擴充及管理基礎架構。擅長處理同步要求/回應工作負載。

Cloud Run 工作

這項服務可與 Cloud Run 服務搭配使用。Cloud Run 工作專為批次處理任務而設計，這類任務必須完成，然後停止執行。非常適合資料處理、ETL、機器學習批次推論，以及任何涉及處理資料集而非處理即時要求的作業。這類服務的主要功能是能夠水平擴展並行執行的容器執行個體 (任務) 數量，以處理批次工作，而且可以由各種事件來源觸發或手動觸發。

主要差異

Cloud Run 服務適用於長時間執行的要求導向應用程式。Cloud Run 工作適用於有時間限制、以工作為導向的批次處理作業，這類作業會執行到完成為止。

建構項目

零售搜尋應用程式

您將：

1. 建立 BigQuery 資料集和資料表，並擷取資料 (Code Vipassana 中繼資料)
2. 建立 Python Cloud Run Functions，實作生成式 AI 功能 (將影片轉換為書籍章節 JSON)
3. 為資料到 AI 管道建立 Python 應用程式 - 從 BigQuery 讀取資料，並叫用 Cloud Run 函式端點來取得洞察資料，然後將內容寫回 BigQuery
4. 建構及容器化應用程式
5. 使用這個容器設定 Cloud Run 工作
6. 執行及監控工作
7. 報表結果

需求條件

- [Chrome](#) 或 [Firefox](#) 瀏覽器
- 已啟用計費功能的 Google Cloud 專案。

2. 事前準備

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

建立專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud 專案。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。
3. 您將使用 [Cloud Shell](#)，這是 Google Cloud 中執行的指令列環境。點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。



4. 連至 Cloud Shell 後，請使用下列指令確認驗證已完成，專案也已設為獲派的專案 ID：

```
gcloud auth list
```

5. 在 Cloud Shell 中執行下列指令，確認 gcloud 指令已瞭解您的專案。

```
gcloud config list project
```

6. 如果未設定專案，請使用下列指令來設定：

```
gcloud config set project <YOUR_PROJECT_ID>
```

7. 啟用必要的 API：按照[這個連結](#)啟用 API。

或者，您也可以使用 gcloud 指令執行這項操作。如要瞭解 gcloud 指令和用法，請參閱[說明文件](#)。

3. 資料庫/倉儲設定

BigQuery 是我們資料管道的骨幹。由於具備無伺服器和高擴充性，因此非常適合儲存輸入資料和存放處理結果。

- **資料儲存：**BigQuery 是我們的資料倉儲。當中會儲存影片網址清單、影片狀態 (例如「待處理」、「處理中」、「已完成」) 和最終生成的內容。這個檔案是影片處理需求的單一可靠資料來源。
- **目的地：**這是指 AI 生成洞察資料的儲存位置，方便下游應用程式查詢，或供人工審查。我們的資料集包含影片工作階段詳細資料，尤其是「Code Vipassana Seasons」內容，這類內容通常會詳細示範技術。
- **來源資料表：**包含記錄的 BigQuery 資料表 (例如 `post_session_labs`)，記錄如下：
- **id：**每個工作階段/資料列的專屬 ID。
- **url：**影片的網址 (例如 YouTube 連結或可存取的雲端硬碟連結)。
- **狀態：**表示處理狀態的字串 (例如：`PENDING`、`PROCESSING`、`COMPLETED`、`FAILED_PROCESSING`)。
- **context：**儲存 AI 生成摘要的字串欄位。
- **資料擷取：**在本情境中，資料是透過 `INSERT` 指令碼擷取至 BigQuery。就我們的管道而言，BigQuery 是起點。

前往 BigQuery 控制台，開啟新分頁並執行下列 SQL 陳述式：

```
--1. Create your dataset for the project

CREATE SCHEMA `<<YOUR_PROJECT_ID>>.cv_metadata`
OPTIONS(
  location = 'us-central1', -- Specify the location (e.g., 'US', 'EU', 'asia-east1')
  description = 'Code Vipassana Sessions Metadata' -- Optional: Add a description
);

--2. Create table

create table cv_metadata.post_session_labs(id STRING, descr STRING, url STRING, context
STRING, status STRING);
```

4. 資料擷取

現在要新增包含商店資料的表格。前往 BigQuery Studio 中的分頁，然後執行下列 SQL 陳述式來插入範例記錄：

```
--Insert sample data

insert into cv_metadata.post_session_labs(id,descr,url) values('10-1','Gen AI to Agents,
where do I begin? Get started with building a single agent application on ADK Python
SDK','https://youtu.be/tyqnQQXpxtI');

insert into cv_metadata.post_session_labs(id,descr,url) values('10-2','Build an E2E multi-
agent kitchen renovation app on ADK in Python with AlloyDB data and multiple
tools','https://youtu.be/RdrMo2lNh0o');

insert into cv_metadata.post_session_labs(id,descr,url) values('10-3','Augment your
multiagent app with tools from MCP Toolbox for AlloyDB','https://youtu.be/9VVNh77Q3ZU?
si=oQ4fhAX59Y3D5iWa');

insert into cv_metadata.post_session_labs(id,descr,url) values('10-4','Build an agentic MCP
client application using MCP Toolbox for BigQuery','https://youtu.be/HmluMag5s20');

insert into cv_metadata.post_session_labs(id,descr,url) values('10-5','Build a travel agent
using ADK & MCP Toolbox for Cloud SQL','https://youtu.be/IWg5CH6ZNs0');

insert into cv_metadata.post_session_labs(id,descr,url) values('10-6','Build an E2E Patent
Analysis Agent using ADK and Advanced Vector Search with
AlloyDB','https://youtu.be/yCXJ3sk3Lxc');

insert into cv_metadata.post_session_labs(id,descr,url) values('10-7','Getting Started with
MCP, ADK and A2A','https://youtu.be/JcQ_DyWc0X0');

update cv_metadata.post_session_labs set status = 'PENDING' where id is not null;
```

5. 建立影片洞察函式

我們必須建立及部署 Cloud Run 函式，實作核心功能，也就是從影片網址建立結構化書籍章節。為了能夠以獨立端點存取這個工具箱工具，我們剛才建立並部署了 Cloud Run 函式。或者，您也可以選擇將此函式納入 Cloud Run 工作實際的 Python 應用程式中：

1. 前往 Google Cloud 控制台的「Cloud Run」頁面。
2. 按一下「編寫函式」。
3. 在「服務名稱」欄位中，輸入描述函式的名稱。服務名稱開頭須為英文字母，且最多只能包含 49 個字元，包括英文字母、數字或連字號。服務名稱結尾不得有連字號，且每個區域和專案的服務名稱不得重複。服務名稱一經設定即無法變更，而且會公開顯示。(*generate-video-insights*)**
4. 在「Region」(區域) 清單中，使用預設值，或選取要部署函式的區域。(選擇 **us-central1**)
5. 在「執行階段」清單中，使用預設值或選取執行階段版本。(選擇 **Python 3.11**)
6. 在「驗證」部分中，選擇「允許公開存取」
7. 按一下「建立」按鈕
8. 系統會建立函式，並載入範本 `main.py` 和 `requirements.txt`
9. 將這些檔案替換為這個專案存放區中的 `main.py` 和 `requirements.txt`

重要事項：請記得在 `main.py` 中，將 `<<YOUR_PROJECT_ID>>` 替換為您的專案 ID。

10. 部署並儲存端點，以便在 Cloud Run 作業的來源中使用。

端點應如下所示 (或類似的內容)：`https://generate-video-insights-<<YOUR_PROJECT_NUMBER>>.us-central1.run.app`

這個 Cloud Run 函式包含哪些內容？

Gemini 2.5 Flash 影片處理

在瞭解及摘要影片內容這項核心工作上，我們採用了 Google 的 Gemini 2.5 Flash 模型。Gemini 模型是強大的多模態 AI 模型，可解讀及處理各種輸入內容，包括文字和影片 (須整合特定服務)。

在我們的設定中，我們並未直接將影片檔案提供給 Gemini，我們改為傳送文字提示，其中包含影片網址，並指示 Gemini 如何分析該網址的 (假設) 影片內容。雖然 Gemini 2.5 Flash 能夠處理多模態輸入內容，但這個特定管道使用的文字提示詞描述了影片性質 (實作實驗室課程)，並要求提供結構化 JSON 輸出內容。這項功能會運用 Gemini 的進階推論和自然語言理解能力，根據提示的背景脈絡推斷及彙整資訊。

Gemini 提示：引導 AI

精心設計的提示對 AI 模型至關重要。我們設計的提示會擷取非常具體的資訊，並以 JSON 格式整理，方便應用程式剖析。


```
PROMPT_TEMPLATE = """
In the video at the following URL: {youtube_url}, which is a hands-on lab session:
Ignore the credits set-up part particularly the coupon code and credits link aspect should
not be included in your analysis or the extraction of context. Also exclude any credentials
that are explicit in the video.
Take only the first 30-40 minutes of the video without throwing any error.
Analyze the rest of the content of the video.
Extract and synthesize information to create a book chapter section with the following
structure, formatted as a JSON string:
1. **chapter_title:** A concise and engaging title for the chapter.
2. **introduction_context:** Briefly explain the relevance of this video segment within a
broader learning context.
3. **what_will_build:** Clearly state the specific task or goal accomplished in this video
segment.
4. **technologies_and_services:** List all mentioned Google Cloud services and any other
relevant technologies (e.g., programming languages, tools, frameworks).
5. **how_we_did_it:** Provide a clear, numbered step-by-step guide of the actions performed.
Include any exact commands or code snippets as they appear in the video. Format code/commands
using markdown backticks (e.g., `my-command`).
6. **source_code_url:** Provide a URL to the source code repository if mentioned or implied.
If not available, use "N/A".
7. **demo_url:** Provide a URL to a demo if mentioned or implied. If not available, use
"N/A".
8. **qa_segment:** Generate 10-15 relevant questions based on the content of this segment,
along with concise answers. Ensure the questions are thought-provoking and test understanding
of the material.
REMEMBER: Ignore the credits set-up part particularly the coupon code and credits link aspect
should not be included in your analysis or the extraction of context. Also exclude any
credentials that are explicit in the video.
Format the entire output as a JSON string. Ensure all keys and string values are enclosed in
double quotes.
Example structure:
...
"""
```

這個提示詞非常具體，引導 Gemini 扮演教育工作者。要求提供 JSON 字串可確保輸出內容結構化，且可供機器讀取。

以下是分析影片輸入內容並傳回其背景資訊的程式碼：

```
def process_videos_batch(video_url: str, PROMPT_TEMPLATE: str) -> str:
    """
    Processes a video URL, generates chapter content using Gemini
    """
    formatted_prompt = PROMPT_TEMPLATE.format(youtube_url=video_url)
    try:

        client = genai.Client(vertexai=True, project='<<YOUR_PROJECT_ID>>', location='us-
centrall1', http_options=HttpOptions(api_version="v1"))
        response = client.models.generate_content(
            model="gemini-2.5-flash",
            contents=formatted_prompt,
        )
        print(response.text)
    except Exception as e:
        print(f"An error occurred during content generation: {e}")
        return f"Error processing video: {e}"
    print(response.text)
    return response.text
```

上述程式碼片段示範了這個用途的核心功能。這項服務會接收影片網址，並透過 Vertex AI 用戶端使用 Gemini 模型分析影片內容，然後根據提示擷取相關洞察資訊。然後，系統會傳回擷取的脈絡資訊，以供後續處理。這代表同步作業，Cloud Run 工作會等待服務完成。

步驟 6 · 約 0 分鐘

6. 管道應用程式開發 (Python)

我們的中央管道邏輯位於應用程式的原始碼中，這些原始碼會容器化為 Cloud Run 作業，負責協調整個平行執行作業。以下是主要部分：

自動化調度管理工具在管理工作流程及確保資料完整性方面的角色：

```

# ... (imports and configuration) ...

def process_batch_from_bq(request_or_trigger_data=None):
    # ... (initial checks for config) ...
    BATCH_SIZE = 5 # Fetch 5 URLs at a time per job instance

    query = f"""
        SELECT url, id
        FROM `{BIGQUERY_PROJECT}`.{BIGQUERY_DATASET}`.{BIGQUERY_TABLE_SOURCE}`
        WHERE status = 'PENDING'
        LIMIT {BATCH_SIZE}
    """

    try:
        logging.info(f"Fetching up to {BATCH_SIZE} pending URLs from BigQuery...")
        rows = bq_client.query(query).result() # job_should_wait=True is default for result()
        pending_urls_data = []
        for row in rows:
            pending_urls_data.append({"url": row.url, "id": row.id})

        if not pending_urls_data:
            logging.info("No pending URLs found. Job finished.")
            return "No pending URLs found. Job finished.", 200

        row_ids_to_process = [item["id"] for item in pending_urls_data]

        # --- Mark as PROCESSING to prevent duplicate work ---
        update_status_query = f"""
            UPDATE `{BIGQUERY_PROJECT}`.{BIGQUERY_DATASET}`.{BIGQUERY_TABLE_SOURCE}`
            SET status = 'PROCESSING'
            WHERE id IN UNNEST(@row_ids_to_process)
        """

        status_update_job_config = bigquery.QueryJobConfig(
            query_parameters=[
                bigquery.ArrayQueryParameter("row_ids_to_process", "STRING",
            values=row_ids_to_process)
            ]
        )

        update_status_job = bq_client.query(update_status_query,
            job_config=status_update_job_config)
        update_status_job.result()
        logging.info(f"Marked {len(row_ids_to_process)} URLs as 'PROCESSING'.")

        # ... (rest of the code for parallel processing and writing) ...

    except Exception as e:
        # ... (error handling) ...

```

上述程式碼片段會先從 BigQuery 來源資料表擷取一批狀態為「PENDING」的影片網址。接著，系統會將這些網址的狀態更新為 BigQuery 中的「PROCESSING」，避免重複處理。

使用 ThreadPoolExecutor 平行處理，並呼叫處理器服務：

```
# ... (inside process_batch_from_bq function) ...

# --- Step 3: Call the external URL Processor Service in parallel ---
processed_results = {}
futures = []

# ThreadPoolExecutor for I/O-bound tasks (HTTP requests to the processor service)
# MAX_CONCURRENT_TASKS_PER_INSTANCE controls parallelism within one job instance.
with ThreadPoolExecutor(max_workers=MAX_CONCURRENT_TASKS_PER_INSTANCE) as executor:
    for item in pending_urls_data:
        url = item["url"]
        row_id = item["id"]
        # Submit the task: call the processor service for this URL
        future = executor.submit(call_url_processor_service, url)
        futures.append((row_id, future))

    # Collect results as they complete
    for row_id, future in futures:
        try:
            content = future.result(timeout=URL_PROCESSOR_TIMEOUT_SECONDS)

            # Check if the processor service returned an error message
            if content.startswith("ERROR:"):
                processed_results[row_id] = {"context": content, "status":
"FAILED_PROCESSING"}
            else:
                processed_results[row_id] = {"context": content, "status":
"COMPLETED"}

        except TimeoutError:
            logging.warning(f"URL processing timed out (service call for row ID
{row_id}). Marking as FAILED.")
            processed_results[row_id] = {"context": f"ERROR: Processing timed out for
'{row_id}'.", "status": "FAILED_PROCESSING"}
        except Exception as e:
            logging.error(f"Exception during future result retrieval for row ID
{row_id}: {e}")
            processed_results[row_id] = {"context": f"ERROR: Unexpected error during
result retrieval for '{row_id}'. Details: {e}", "status": "FAILED_PROCESSING"}
```

這部分程式碼會運用 ThreadPoolExecutor，平行處理擷取的影片網址。針對每個網址，程式碼會提交工作，以非同步方式呼叫 Cloud Run 服務（網址處理器）。這樣一來，Cloud Run 作業就能同時處理多部影片，進而提升整體管道效能。程式碼片段也會處理處理器服務可能發生的逾時和錯誤。

從 BigQuery 讀取資料，以及將資料寫入 BigQuery

與 BigQuery 的核心互動包括擷取待處理的網址，然後使用處理結果更新這些網址。

```
# ... (inside process_batch_from_bq) ...
BATCH_SIZE = 5
query = f"""
    SELECT url, id
    FROM `{BIGQUERY_PROJECT}.{BIGQUERY_DATASET}.{BIGQUERY_TABLE_SOURCE}`
    WHERE status = 'PENDING'
    LIMIT {BATCH_SIZE}
"""

rows = bq_client.query(query).result()

pending_urls_data = []
for row in rows:
    pending_urls_data.append({"url": row.url, "id": row.id})
# ... (rest of fetching and marking as PROCESSING) ...
```

將結果寫回 BigQuery :

```

# --- Step 4: Write results back to BigQuery ---
    logging.info(f"Writing {len(processed_results)} results back to BigQuery...")
    successful_updates = 0
    for row_id, data in processed_results.items():
        if update_bq_row(row_id, data["context"], data["status"]):
            successful_updates += 1

    logging.info(f"Finished processing. {successful_updates} out of
{len(processed_results)} rows updated successfully.")
    # ... (return statement) ...

# --- Helper to update a single row in BigQuery ---
def update_bq_row(row_id, context, status="COMPLETED"):
    """Updates a specific row in the target BigQuery table."""
    # ... (checks for config) ...

    update_query = f"""
        UPDATE `{BIGQUERY_PROJECT}`.{BIGQUERY_DATASET}`.{BIGQUERY_TABLE_TARGET}`
        SET
            context = @context,
            status = @status
        WHERE id = @row_id
    """

    # Correctly defining query parameters for the UPDATE statement
    job_config = bigquery.QueryJobConfig(
        query_parameters=[
            bigquery.ScalarQueryParameter("context", "STRING", value=context),
            bigquery.ScalarQueryParameter("status", "STRING", value=status),
            # Assuming 'id' column is STRING. Adjust if it's INT64.
            bigquery.ScalarQueryParameter("row_id", "STRING", value=row_id)
        ]
    )

    try:
        update_job = bq_client.query(update_query, job_config=job_config)
        update_job.result() # Wait for the job to complete
        logging.info(f"Successfully updated BigQuery row ID {row_id} with status {status}.")
        return True
    except Exception as e:
        logging.error(f"Failed to update BigQuery row ID {row_id}: {e}")
        return False

```

上述程式碼片段著重於 Cloud Run 工作與 BigQuery 之間的資料互動。從來源資料表擷取一批「待處理」的影片網址和 ID。處理完網址後，這個程式碼片段會示範如何使用 UPDATE 查詢，將擷取的內容和狀態（「COMPLETED」或「FAILED_PROCESSING」）寫回目標 BigQuery 資料表。這個程式碼片段會完成資料處理迴圈。此外，這個檔案也包含 update_bq_row 輔助函式，說明如何定義更新陳述式的參數。

應用程式設定

應用程式的結構為單一 Python 指令碼，將會容器化。這項工具會運用 Google Cloud 用戶端程式庫和 functions-framework，定義進入點。

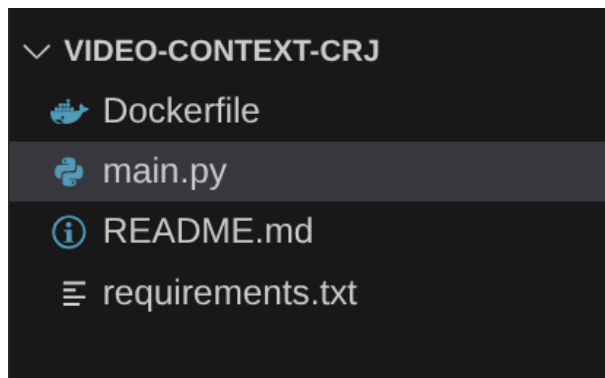
- 依附元件：google-cloud-bigquery、requests
- 設定：所有重要設定 (BigQuery 專案/資料集/資料表、網址處理器服務網址) 都是從環境變數載入，因此應用程式可攜且安全
- 核心邏輯：process_batch_from_bq 函式會自動調度整個工作流程
- 外部服務整合：call_url_processor_service 函式會處理與獨立 Cloud Run 服務的通訊
- BigQuery 互動：bq_client 用於擷取網址和更新結果，並適當處理參數
- 平行處理：concurrent.futures.ThreadPoolExecutor 會管理對外部服務的並行呼叫
- 進入點：名為 main.py 的 Python 程式碼會做為進入點，啟動批次處理作業。

現在來設定應用程式：

1. 首先，請前往 Cloud Shell 終端機並複製存放區：

```
git clone https://github.com/AbiramiSukumaran/video-context-crj
```

2. 前往 Cloud Shell 編輯器，您會看到新建立的資料夾 **video-context-crj**
3. 刪除下列項目，因為這些步驟已在先前的章節中完成：
4. 刪除 Cloud_Run_Function 資料夾
5. 前往專案資料夾 **video-context-crj**，您應該會看到專案結構：



步驟 7 · 約 0 分鐘

7. 設定 Dockerfile 和容器化

如要將這項邏輯部署為 Cloud Run 作業，我們需要將其容器化。容器化是將應用程式程式碼、依附元件和執行階段封裝成可攜式映像檔的程序。

請務必在 Dockerfile 中將預留位置 (粗體文字) 替換為您的值：

```
# Use an official Python runtime as a parent image
FROM python:3.12-alpine

# Set the working directory in the container
WORKDIR /app

# Copy the requirements file into the container
COPY requirements.txt .

# Install any needed packages specified in requirements.txt
RUN pip install --trusted-host pypi.python.org -r requirements.txt

# Copy the rest of the application code
COPY . .

# Define environment variables for configuration (these will be overridden during deployment)
ENV BIGQUERY_PROJECT="YOUR-project"
ENV BIGQUERY_DATASET="YOUR-dataset"
ENV BIGQUERY_TABLE_SOURCE="YOUR-source-table"
ENV URL_PROCESSOR_SERVICE_URL="ENDPOINT FOR VIDEO PROCESSING"
ENV BIGQUERY_TABLE_TARGET = "YOUR-destination-table"

ENTRYPOINT ["python", "main.py"]
```

上述 Dockerfile 片段會定義基礎映像檔、安裝依附元件、複製程式碼，並設定指令，使用 functions-framework 執行應用程式，以及正確的目標函式 (process_batch_from_bq)。然後將此映像檔推送至 Artifact Registry。

容器化

如要將其容器化，請前往 Cloud Shell 終端機並執行下列指令 (請記得將 <<YOUR_PROJECT_ID>> 預留位置替換為您的專案 ID)：

```
export CONTAINER_IMAGE="gcr.io/<<YOUR_PROJECT_ID>>/batch-url-processor-orchestrator:latest"

gcloud builds submit --tag $CONTAINER_IMAGE .
```

建立容器映像檔後，您應該會看到以下輸出內容：

```

PUSH
Pushing gcr.io/[redacted]/batch-url-processor-orchestrator:latest
The push refers to repository [gcr.io/[redacted]batch-url-processor-orchestrator]
35437f28d7b6: Preparing
22ade70cff12: Preparing
104ccca7c33f: Preparing
37632fbd4ffb: Preparing
4774e58ebefa: Preparing
fcdfe0128: Preparing
41c84462c2c0: Preparing
418dccb7d85a: Preparing
fcdfe0128: Waiting
41c84462c2c0: Waiting
418dccb7d85a: Waiting
4774e58ebefa: Layer already exists
fcdfe0128: Layer already exists
41c84462c2c0: Layer already exists
418dccb7d85a: Layer already exists
37632fbd4ffb: Pushed
35437f28d7b6: Pushed
104ccca7c33f: Pushed
22ade70cff12: Pushed
latest: digest: sha256:baa381d2393662fc1eb00d4552b3f7abd876d0a9c05b5ddc12cdaf767ee165c0 size: 1991
DONE
-----
ID                                CREATE_TIME                     IMAGES                           DURATION  SOURCE
STATUS
a0f3ae4b-3055-40bd-867b-f940db65e448  2025-08-28T10:32:49+00:00  41S                             gs://[redacted]_cloudbuild/source/1
756377168.760584-9ba470b53d2e44589d676ed17e852f90.tgz  gcr.io/[redacted]batch-url-processor-orchestrator (+1

```

容器現已建立並儲存在 Artifact Registry 中。現在可以前往下一個步驟。

步驟 8 · 約 0 分鐘

8. 建立 Cloud Run 工作

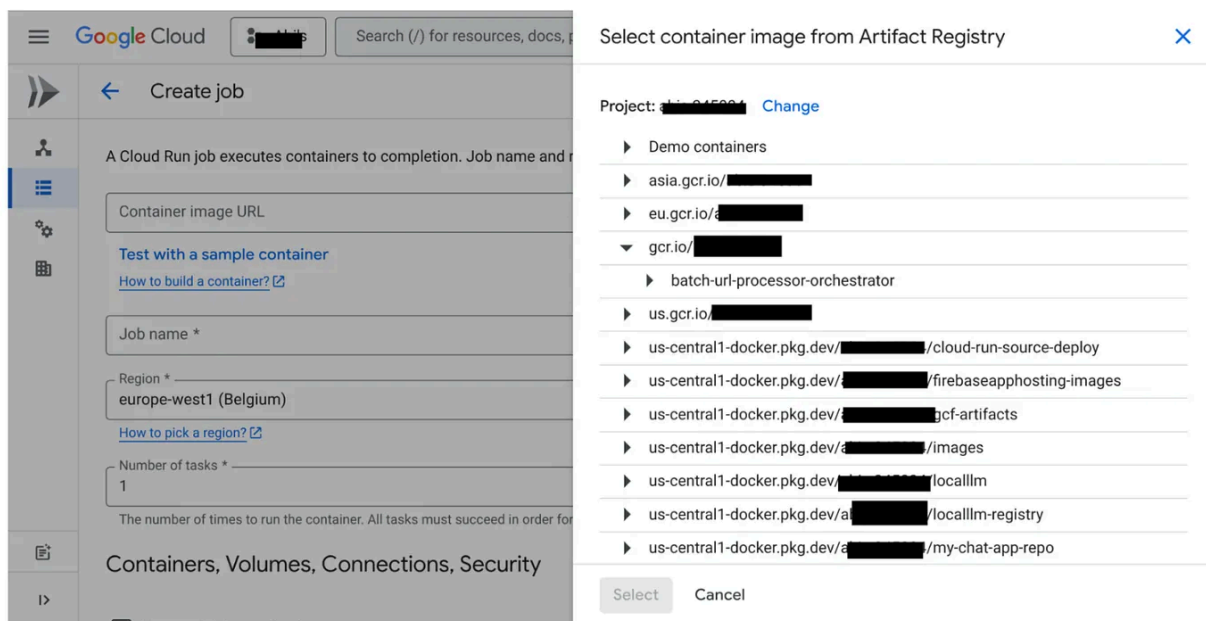
部署工作時，需要先建構容器映像檔，然後建立 Cloud Run Job 資源。

我們已建立容器映像檔，並儲存在 Artifact Registry 中。現在來建立工作。

1. 前往 [Cloud Run Jobs](#) 控制台，然後按一下「Deploy Container」(部署容器)：



2. 選取我們剛才建立的容器映像檔：



3. 輸入其他設定詳細資料，如下所示：

Job name *
batch-url-processor-orchestrator-latest

Region *
us-central1 (Iowa)
[How to pick a region?](#)

Number of tasks *
20
The number of times to run the container. All tasks must succeed in order for a job to succeed.

4. 請依下列方式設定工作負荷能力：

Task capacity

Task timeout *
10

Maximum running time for one task (maximum 10080 minutes). Values greater than 1440 are in Preview.

Time unit *
minute

Number of retries per failed task *
3

Parallelism

The maximum number of tasks running at the same time.

☐ Run as many tasks concurrently as possible

Maximum parallelism depends on region. See the [quotas page](#) for details.

☒ Limit the number of concurrent tasks

Use this option to limit the number of concurrent requests to backing resources such as databases or file systems.

Custom parallelism limit

1

Must be no higher than 10000 for selected CPU and memory on the region. 0 is maximum parallelism.

由於我們有資料庫寫入作業，且程式碼中已處理平行化 (max_instances 和工作並行)，因此我們會將並行工作數設為 1。但您可以視需求增加。目標是讓工作根據設定完成，並在平行處理中設定並行層級。

5. 點選 [建立]

系統會成功建立 Cloud Run 工作。

運作方式

工作容器執行個體隨即啟動。這項函式會查詢 BigQuery，取得一小批 (BATCH_SIZE) 標示為「待處理」的網址。系統會立即將這些擷取網址的狀態更新為 BigQuery 中的「PROCESSING」，防止其他工作例項選取這些網址。這個函式會建立 ThreadPoolExecutor，並為批次中的每個網址提交工作。每個工作都會呼叫 call_url_processor_service 函式。call_url_processor_service 要求完成 (或逾時/失敗) 後，系統會收集結果 (AI 生成的內容或錯誤訊息)，並對應回原始 row_id。批次的所有工作完成後，這項工作會逐一檢查收集到的結果，並更新 BigQuery 中每個對應資料列的內容和狀態欄位。如果成功，工作執行個體會正常結束。如果遇到未處理的錯誤，就會引發例外狀況，可能導致 Cloud Run Jobs 觸發重試 (視工作設定而定)。

Cloud Run Jobs 的適用情境：自動化調度管理

這正是 Cloud Run Jobs 的優勢所在。

無伺服器批次處理：我們取得受管理的基礎架構，可視需要啟動任意數量的容器執行個體 (最多 MAX_INSTANCES)，同時處理資料。

平行處理控制項：我們定義 MAX_INSTANCES (整體可平行執行的工作數量) 和 TASK_CONCURRENCY (每個工作執行個體平行執行的作業數量)。這樣一來，您就能精細控管處理量和資源用量。

容錯能力：如果工作執行個體在執行途中失敗，您可以將 Cloud Run Jobs 設為重試整個工作或特定工作，確保資料處理作業不會中斷。

簡化架構：直接在 Job 中協調 HTTP 呼叫，並使用 BigQuery 管理狀態，避免設定及管理 Pub/Sub、主題、訂閱項目和確認邏輯的複雜性。

MAX_INSTANCES 與 TASK_CONCURRENCY：

MAX_INSTANCES: 整個工作執行作業可同時執行的工作執行個體總數。這是處理大量網址時的主要平行處理槓桿。

TASK_CONCURRENCY:單一作業執行個體執行的平行作業 (呼叫處理器服務) 數量。這有助於讓一個執行個體的 CPU/網路達到飽和。

9. 執行及監控 Cloud Run 工作

影片中繼資料

在點選「執行」前，先查看資料狀態。

前往 BigQuery Studio 並執行下列查詢：

```
Select id, descr, url, status from cv_metadata.post_session_labs where status = 'PENDING'
```

1 select id, descr, url, status from bg_gemini_test2.post_session_labs where status = 'PENDING'

2

Completed

Query results

Save results Open in

Job information Results Visualisation JSON Execution details Execution graph

Row	id	descr	url	status
1	9-5	Toystore App - Part 2 (Web app,...	https://youtu.be/EEe_7jsEWV8	PENDING
2	10-6	Build an E2E Patent Analysis Agent using ADK and Advanced Vector Search with	https://youtu.be/yCXJ3sk3Lxc	PENDING

我們提供幾個含有影片網址且狀態為「待處理」的範例記錄。我們的目標是根據提示中說明的格式，在「context」欄位中填入影片洞察資料。

工作觸發條件

請點選 Cloud Run Jobs 控制台中的「EXECUTE」按鈕，執行工作。您應該可以在控制台中查看工作的進度 and 狀態：

batch-url-processor-orchestrator-new-rv6sw

Tasks overview (20): 20 Succeeded 0 Failed 0 Running

Tasks Containers Volumes Networking Security YAML

Filter Filter tasks

Task	Last exit code	Retries	Start time	End time	
0	0	0	Aug 28, 2025, 10:56:51 AM	Aug 28, 2025, 10:57:59 AM	View logs
1	0	0	Aug 28, 2025, 10:59:05 AM	Aug 28, 2025, 11:00:03 AM	View logs
2	0	0	Aug 28, 2025, 11:01:17 AM	Aug 28, 2025, 11:02:24 AM	View logs
3	0	0	Aug 28, 2025, 11:03:26 AM	Aug 28, 2025, 11:04:12 AM	View logs
4	0	0	Aug 28, 2025, 11:05:31 AM	Aug 28, 2025, 11:06:27 AM	View logs
5	0	0	Aug 28, 2025, 11:07:34 AM	Aug 28, 2025, 11:08:43 AM	View logs

您可以在「OBSERVABILITY」中的「LOGS」標記中，監控工作和工作步驟，以及查看其他詳細資料。

10. 結果分析

工作完成後，您應該就能在表格中看到每個影片網址的背景資訊已更新：

2

select id, descr, url, context from bg_gemini_test2.post_session_labs

3

Completed

Query results

Save results

Job information

Results

Visualisation

JSON

Execution details

Execution graph

descr	url	context
		Cloud Run", "introduction_context": "This hands-on lab introduces
Build an E2E Patent Analyse Agent using	https://youtu.be/yCXJ3sk3Lxc	```json {

輸出背景資訊 (適用於其中一筆記錄)

```

{
  "chapter_title": "Building a Travel Agent with ADK and MCP Toolbox",
  "introduction_context": "This chapter section is derived from a hands-on lab session
focused on building a travel agent. It details the process of integrating various Google
Cloud services and tools to create an intelligent agent capable of querying a database and
interacting with users.",
  "what_will_build": "The goal is to build and deploy a travel agent that can answer user
queries about hotels using the Agent Development Kit (ADK) and the MCP Toolbox for Databases,
connecting to a PostgreSQL database.",
  "technologies_and_services": [
    "Google Cloud Platform",
    "Cloud SQL for PostgreSQL",
    "Agent Development Kit (ADK)",
    "MCP Toolbox for Databases",
    "Cloud Shell",
    "Cloud Run",
    "Python",
    "Docker"
  ],
  "how_we_did_it": [
    "Provision a Cloud SQL instance for PostgreSQL with the 'hoteldb-instance'.",
    "Prepare the 'hotels' database by creating a table with relevant schema and populating it
with sample data.",
    "Set up the MCP Toolbox for Databases by downloading and configuring the necessary
components.",
    "Install the Agent Development Kit (ADK) and its dependencies.",
    "Create a new agent using the ADK, specifying the model (Gemini 2.0-flash) and backend
(Vertex AI).",
    "Modify the agent's code to connect to the PostgreSQL database via the MCP Toolbox.",
    "Run the agent locally to test its functionality and ability to interact with the
database.",
    "Deploy the agent to Cloud Run for cloud-based access and further testing.",
    "Interact with the deployed agent through a web console or command line to query hotel
information."
  ],
  "source_code_url": "N/A",
  "demo_url": "N/A",
  "qa_segment": [
    {
      "question": "What is the primary purpose of the MCP Toolbox for Databases?",
      "answer": "The MCP Toolbox for Databases is an open-source MCP server designed to help
users develop tools faster, more securely, and by handling complexities like connection
pooling, authentication, and more."
    },
    {
      "question": "Which Google Cloud service is used to create the database for the travel
agent?",
      "answer": "Cloud SQL for PostgreSQL is used to create the database."
    },
    {
      "question": "What is the role of the Agent Development Kit (ADK)?",
      "answer": "The ADK helps build Generative AI tools that allow agents to access data in

```



```
a database. It enables agents to perform actions, interact with users, utilize external
tools, and coordinate with other agents."
    },
    {
      "question": "What command is used to create the initial agent application using ADK?",
      "answer": "The command `adk create hotel-agent-app` is used to create the agent
application."
    },
    ....
  ],
}
```

現在，您應該可以驗證這個 JSON 結構，以用於更進階的代理程式用途。

為什麼要採用這種做法？

這種架構可帶來顯著的策略優勢：

- 成本效益：無伺服器服務只會針對實際用量收費。不使用時，Cloud Run 工作會將資源調度率降至零。
 - 擴充性：調整 Cloud Run Job 執行個體和並行設定，輕鬆處理數以萬計的網址。
 - 靈活度：只要更新所含應用程式及其服務，即可快速開發及部署新的處理邏輯或 AI 模型。
 - 降低營運成本：無須修補或管理伺服器，Google 會處理基礎架構。
 - 普及 AI：讓使用者不必具備深厚的機器學習作業專業知識，也能存取進階 AI 處理功能，執行批次工作。
-

11. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本文章所用資源的費用，請按照下列步驟操作：

1. 前往 Google Cloud 控制台的[資源管理員](#)頁面。
 2. 在專案清單中選取要刪除的專案，然後點按「刪除」。
 3. 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關機) 即可刪除專案。
-

12. 恭喜

恭喜！您以 Cloud Run Jobs 為架構基礎，並運用 BigQuery 的資料管理功能和外部 Cloud Run Service 的 AI 處理功能，建構出具備高擴充性、符合成本效益且易於維護的系統。這種模式可將處理邏輯分離，允許平行執行作業，不必使用複雜的基礎架構，並大幅縮短取得洞察資料的時間。

建議您探索 Cloud Run 工作，滿足自己的批次處理需求。無論是擴充 AI 分析、執行 ETL 管道，還是執行週期性資料工作，這種無伺服器方法都能提供強大且有效率的解決方案。如要自行開始使用，請[參閱這篇文章](#)。

如果您想以無伺服器和代理程式的方式建構及部署所有應用程式，請報名參加 [Code Vipassana](#)，瞭解如何加速開發以資料為基礎的生成式代理程式應用程式！
